Excelerate Connect

Week 3 Changelog & Documentation — Team 7

Repository: github.com/KrishTrivedi025/excelerate_connect
Submission window: Jul 4, 2026 – Jul 27, 2026, 11:59 PM

Status note: items marked [Done] are merged on main today. Items marked [In progress] are agreed and actively being built, finishing before the submission deadline — update this note once they land.

1. Screens now fetching data from a mock API

Both required screens moved off static, hardcoded data onto a mock async service (`OpportunityService`) that simulates a real network call — a delay, then either the data or a simulated failure. No new dependencies or JSON parsing were needed; this satisfies the requirement's own stated alternative to a JSON file.

Program Listing *[Done]*

- `OpportunityService.fetchOpportunities()` — fetched via a `FutureBuilder`, driving a loading spinner, an error+retry card, or the live list.
- Pull-to-refresh re-triggers the same fetch without leaving the screen.
- Search and category filters stay live and interactive the whole time — they operate on whatever data has already arrived, not a frozen snapshot.

Program Details *[In progress]*

- `OpportunityService.fetchOpportunityById()` (already built for Program Listing) is being wired into Program Details so it re-fetches its own program fresh every time the screen opens, rather than trusting the copy handed over during navigation. In practice: if an admin edits a program's details, re-opening that program's page will pick up the change without needing the list re-fetched too.
- Same loading / error+retry treatment as Program Listing, using the shared `ErrorRetryCard` widget.

2. Form added, and how it works

Feedback screen — rebuilt this week *[Done]*

Reachable from Program Details once a learner has registered (the primary action button swaps from “Register Now” to “Give Feedback” on success). Fields:

- **Mood slider** — a 5-step drag control (not a plain star rating) with a caption and haptic feedback on each step.
- **“What helped most” chips** — generated from the actual course's own skill list, not a hardcoded set, so it's correct for any course.
- **Recommend** — a Yes / Maybe / No single-select.

- **Name** and **Email** — both required; email is checked against a regex and rejected if it isn't a valid address.
- **Comments** — optional, user-resizable text area.
- On submit: validates → shows a loading state → a success panel with a countdown ring that visibly drains over 5 seconds → returns automatically to Program Listing.

Registration screen [Also validated]

Required first/last name, a native date-of-birth picker, an optional gender selector, a required “how did you hear about this” source, and a regex-checked email — all enforced before submission is allowed, with the same loading-then-success-panel pattern as Feedback.

3. Loading & error-handling features

Shipped [Done]

- **OpportunityService** — the shared mock async layer Program Listing (and shortly Program Details) call into; includes a documented `debugForceError` toggle used to demo the failure path on demand (defaults to off).
- **ErrorRetryCard** — one reusable error widget (icon, message, “Try Again” button) instead of a separate one-off per screen.
- **Submit-and-confirm pattern** — Registration and Feedback both show a loading state while “submitting,” then a success panel with a countdown ring that visibly drains before auto-returning — so the user always knows something is about to happen, not just that it happened.

In progress [In progress]

- **Branded loading indicator** — replacing the generic spinner with Excelerate's own animated mark for every loading state: Login, Program Listing, and Program Details.
- **Post-login and post-registration loading** — a brief branded loading screen bridging the gap between submitting and the next screen appearing, instead of an instant cut.

Demo video

[VIDEO LINK PLACEHOLDER — add your Drive/YouTube link here before submitting]

Optional per the assignment, but recommended: a 2–3 minute walkthrough of Program Listing pulling live (mocked) data and a form submission end to end.